PSI 2025-2026 – Objectif 2 : Planification optimale de tournées

Michaël Soulignac

ESILV Nantes

Rappel : Contexte du projet

Problème du voyageur de commerce (*Traveling Salesman Problem – TSP*)

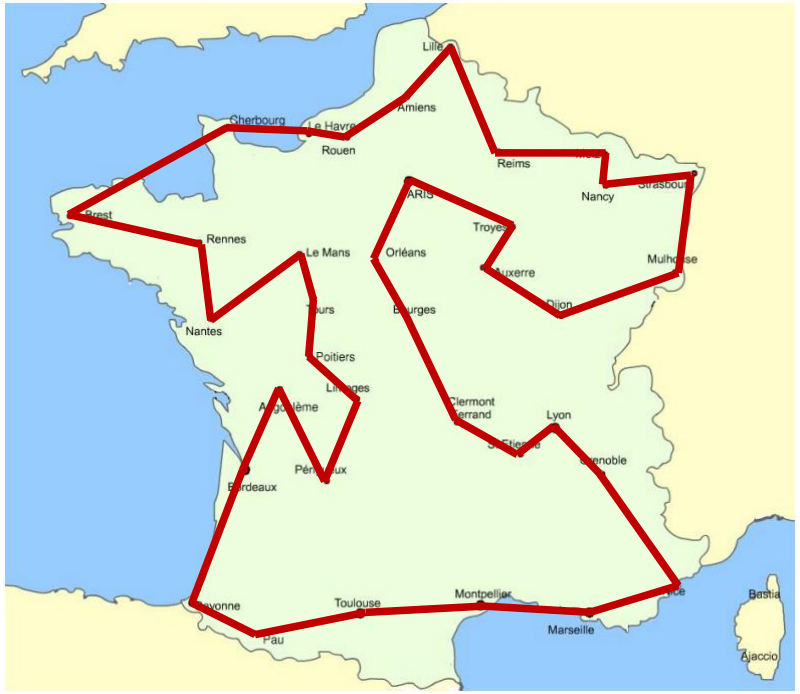
> Un voyageur de commerce doit visiter une et une seule fois un nombre fini de villes et revenir à son point d'origine.
> Trouvez l'ordre de visite des villes qui minimise la distance totale parcourue par le voyageur

Posé pour la première fois par William Rowan Hamilton en 1859

TSP : Exemple

36 villes à visiter

Tournée optimale



Formalisation d'un TSP

Soit G un graphe d'ordre n avec :

- $V(G)$ l'ensemble des nœuds, chaque nœud modélisant une ville
- $E(G)$ l'ensemble des arcs. Le poids $w(e)$ associé à chaque arc $e \in E(G)$ modélise au coût de déplacement entre deux villes.

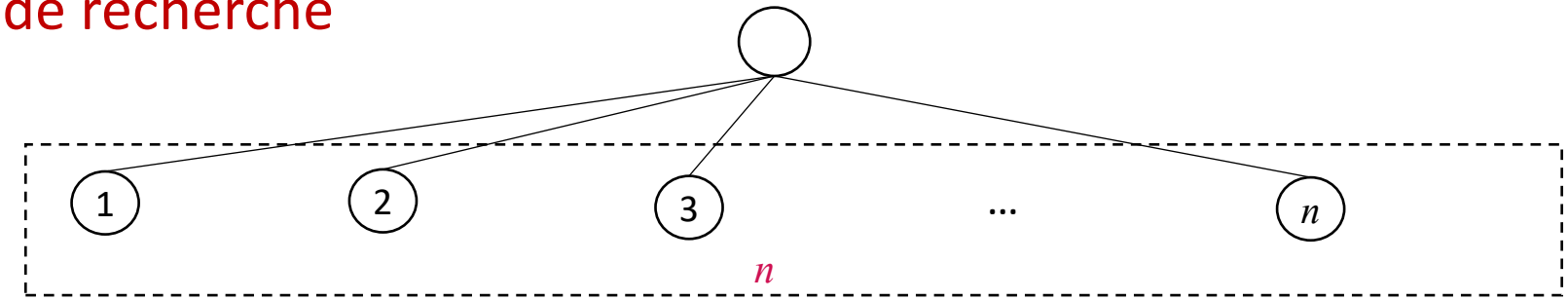
Le problème du voyageur de commerce consiste à calculer un **cycle hamiltonien de coût minimal** dans G.

C'est un problème **NP-complet** : il n'existe pas d'algorithme permettant de trouver une solution en temps polynomial.

Force brute: principe

Enumérer toutes les tournées et choisir celle de coût minimal

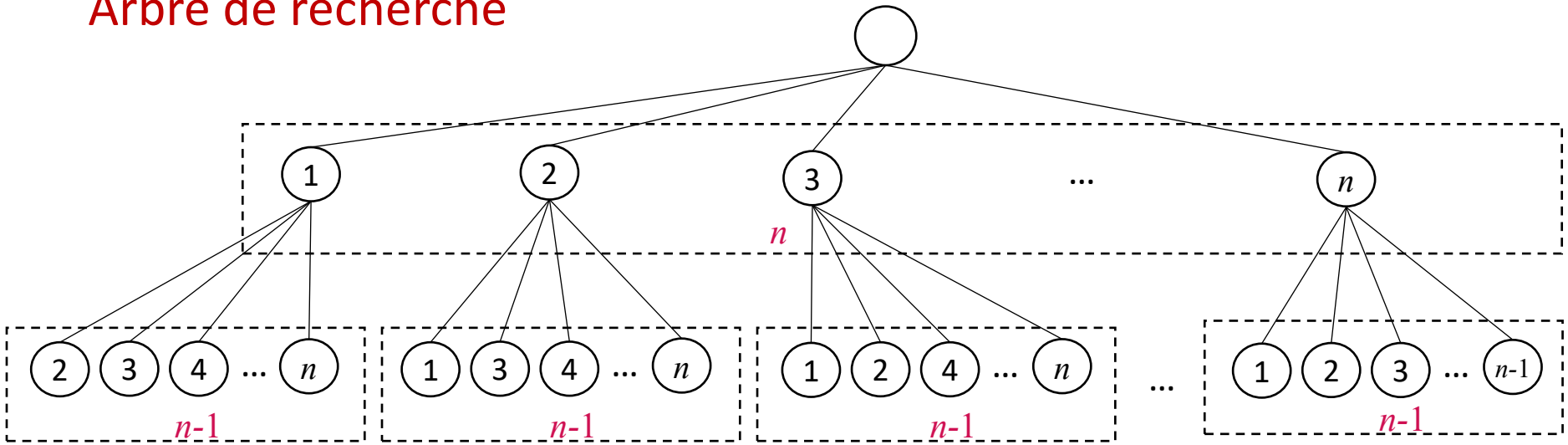
Arbre de recherche



Force brute: principe

Enumérer toutes les tournées et choisir celle de coût minimal

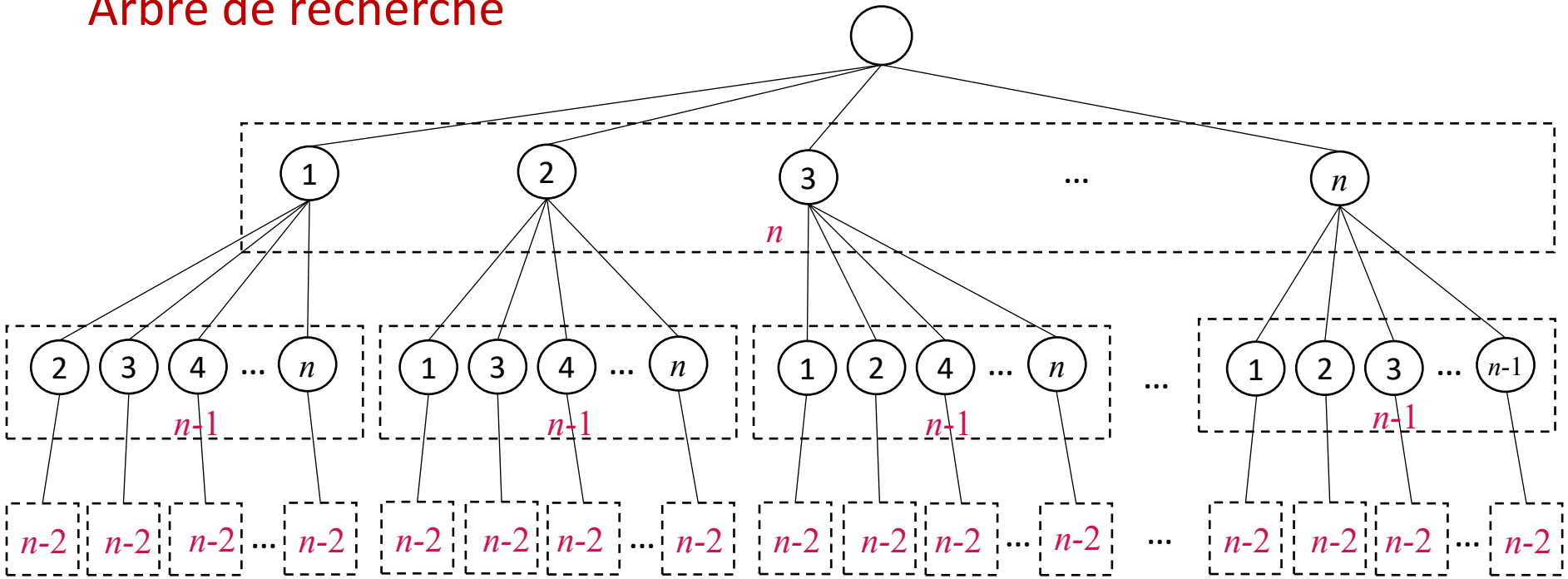
Arbre de recherche



Force brute: principe

Enumérer toutes les tournées et choisir celle de coût minimal

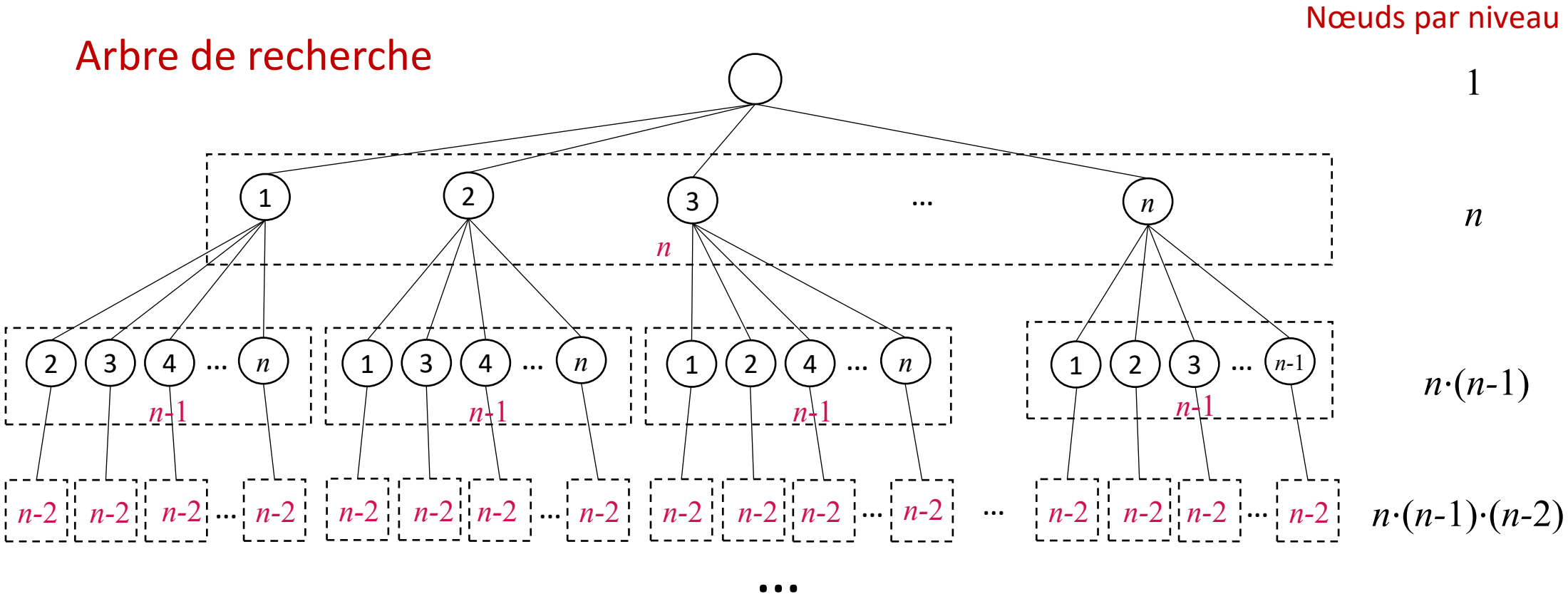
Arbre de recherche



Force brute: principe

Enumérer toutes les tournées et choisir celle de coût minimal

Arbre de recherche



Force brute: complexité temporelle

Niveau	Nombre de nœuds	Peut être récrit comme
0 (racine)	1	$n! / n!$
1	n	$n! / (n-1)!$
2	$n \cdot (n-1)$	$n! / (n-2)!$
3	$n \cdot (n-1) \cdot (n-2)$	$n! / (n-3)!$
…	…	…
n-2	$n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 5 \cdot 4 \cdot 3$	$n! / 2!$
n-1	$n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 4 \cdot 3 \cdot 2$	$n! / 1!$
n	$n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 3 \cdot 2 \cdot 1$	$n! / 0!$

Nombre total de nœuds:

$$N(n) = \sum_{k=0}^{n} \frac{n!}{(n-k)!}$$

$$= n! \sum_{m=0}^{n} \frac{1}{m!} \quad \text{avec } m = n - k$$

$N(n) \to e \cdot n!$ quand $n \to \infty$

$\Rightarrow$ Complexité temporelle : $O(n!)$

…

Force brute: complexité temporelle

Exemples pour quelques valeurs de n :

n	Nombre exact de nœuds	Approximation avec $e \cdot n!$
5	326	≈ 326.19
6	1 957	$\approx 1\,957.16$
10	9 864 101	$\approx 9\,864\,101.10$
50	82 674 076 879 277 258 572 496 581 009 301 773 302 984 486 449 338 756 300 825 298 501	$\approx 8.267407687927725 \times 10^{64}$

...

L'algorithme de Little

Elague **fortement** l'arbre de recherche par *branch & bound* :

- Calcule une borne inférieure pour le coût de la tournée par réduction de la matrice des coûts
- Sépare de l'espace de recherche en incluant ou excluant des trajets choisis selon leur regret
- Coupe toutes les branches ne pouvant pas améliorer la solution

$\Rightarrow$ Complexité temporelle exponentielle en pratique : $\mathrm{O}(c^n)$

La base c dépend des propriétés du problème d'entrée, notamment la structure des coûts.

Objectif 2

Implémenter l'algorithme de Little en C#

Sous-tâches :

1. Compréhension de l'algorithme (exemple à la main)
2. Implémentation des étapes clés
3. Implémentation de l'algorithme dans son ensemble

Eléments fournis

Eléments fournis

Pour vous aider dans cet objectif, les classes suivantes vous sont fournies :

- `Little` : classe modélisant l'algorithme de Little (à compléter)
- `Tour` : classe modélisant une tournée (à compléter)
- `LittleTests` : campagne de tests vérifiant le bon fonctionnement de `Little` (étapes clés et algorithme entier)

Eléments à compléter

- Little : 5 méthodes imposées :
  - Constructeur
  - 3 étapes clés (réduction, regrets, sous-tournées)
  - Calcul des tournées
- Tour : 4 méthodes imposées :
  - 3 propriétés
  - test d'appartenance d'un trajet à la tournée
  - Le constructeur est laissé libre et dépend de vos choix de modélisation

Modifications autorisées et interdites

- Interdit :
  - Renommer les classes ou les fichiers fournis
  - Modifier la signature des méthodes imposées (nom, types)
- Autorisé (et nécessaire !):
  - Implémenter le corps des méthodes imposées
  - Ajouter des attributs et des méthodes dans les classes fournies

  ⇒ En particulier : ne pas hésiter à enrichir Matrix et Graph avec de nouvelles fonctionnalités – du moment que cela ne « casse » pas les tests unitaires fournis (non-régression)
  - Ajouter d'autres classes, et les réutiliser dans les classes fournies

Ajouter vos propres classes

- En dehors des méthodes imposées, la modélisation est libre : ajoutez toutes les classes que vous jugerez pertinentes dans votre code et votre diagramme UML

⇒ Exemple : un regret peut être modélisé par une classe.

- La qualité de votre modélisation sera évaluée dans l'objectif 3

Critères d’évaluation

Critères d'évaluation

- Implémentation fonctionnelle = note de l'objectif 2

⇒ Evalué après le rendu de l'objectif 2 par des tests unitaires

- Modélisation = partie de la note de l'objectif 3 (rapport)

⇒ Compléter le diagramme de classes démarré dans l'objectif 1, avec les nouvelles classes introduites dans cet objectif

⇒ Non évalué tout de suite, mais à anticiper (préparer les éléments)

Note de l'objectif 2

- Campagne de tests unitaires `LittleTests`
- Barème :

Test unitaire	Elément testé	Points
ReductionTest	Réduction de la matrice de coûts	2
RegretTest	Calcul du regret maximal	2
ForbiddenSegmentsTest_ReverseSegment	Détection d'un trajet parasite dans un cas simple (trajet inverse)	2
ForbiddenSegmentsTest_Subtour	Détection d'un trajet parasite dans un cas complexe (sous-tournée)	4
LittleSymetricCosts	Calcul de tournée par l'algorithme de Little sur un problème symétrique (cours)	5
LittleAsymetricCosts	Calcul de tournée par l'algorithme de Little sur un problème asymétrique (TD)	5

Note de l'objectif 2

- Campagne de tests unitaires `LittleTest`
- Si le code ne compile pas ⇒ Note = 0

⇒ Aucune surprise sur la note, nous la connaîtrez avant de rendre votre code